Unit II: Synchronization and Coordination

Distributed Systems - Time, States, Coordination

Need of Synchronization in DS

- It is important that multiple processes do not simultaneously access a shared resource, such as printer, but instead cooperate in granting each other temporary exclusive access.
- Multiple processes may sometimes need to agree on the ordering of events, such as whether message *ml* from process *P* was sent before or after message *m2* from process Q.

Problem with Physical Clocks

- Multiple physical clocks are generally considered desirable, which yields two problems:
- (1) How do we synchronize them with real world clocks.
- (2) How do we synchronize the clocks with each other?

In distributed systems, there is no single global clock.

- A distributed system consists of multiple independent computers (nodes) connected by a network.
- Each machine has its own local clock, but due to hardware differences and network delays, these clocks drift (move at slightly different speeds).
- Unlike in a single system, there is no shared physical clock to keep all nodes in perfect sync.

Time and Global States

- Logical Clocks: Order events without physical time
- Vector Clocks: Capture causality among events
- Global State: Snapshot of distributed system's state

Synchronizing Physical Clocks

- Cristian's Algorithm: Uses time server
- Berkeley Algorithm: Average of all clocks

Berkeley Algorithm

Problem

- In distributed systems, there is no global clock.
- Each machine has its own local clock drift occurs (clocks differ over time).
- Need to synchronize clocks so that all processes have nearly the same time

Berkeley Algorithm

Working of Berkeley Algorithm

- One machine is chosen as the time coordinator (master).
- Others act as slaves.
- The master polls all slave clocks to get their time.
- The master:
  - Collects time differences from slaves.
  - Discards extreme values (outliers).
  - Calculates the average time difference.
  - Sends the adjustment (offset) to each slave.
- Each machine adjusts its clock gradually (not by resetting, to avoid sudden jumps).

Berkeley Algorithm

- Master sends request to all slaves asking for their clock values.
- Each slave replies with its local time.
- Master collects times, computes average clock difference.
- Master sends time adjustment instructions (advance or delay) to each slave.
- All clocks are synchronized close to the master’s time.

Cristian's Algorithm

Problem

- In a distributed system, each machine has its own local clock no global clock.
- Need to synchronize with a reliable time source (time server).
- Cristian's Algorithm provides a way to synchronize using round-trip message delays.

Cristian's Algorithm

Working

- One node is chosen as the time server (has accurate clock, e.g., atomic/GPS).
- A client process requests the current time from the server.
- Server responds with its current time T.
- Client sets its clock to T + (RTT/2) where:
  - RTT = Round Trip Time (measured by client).
  - (RTT/2) accounts for network delay (assuming symmetric delays).

Cristian's Algorithm

- Client notes local time t0 before sending request.
- Server receives and replies with its time T.
- Client receives reply at time t1.
- Client estimates network delay = (t1 – t0)/2.
- Client sets its local clock = T + (t1 – t0)/2.

Consensus and Agreement

- Consensus Problem: All nodes agree on a value
- Byzantine Fault Tolerance: Handles malicious nodes
- Paxos and Raft: Popular consensus protocols